AI-Powered Smart Campus Safety Simulation

Brandy Griffin

ITAI 3377

This notebook demonstrates a simulated AI-powered campus safety system that detects suspicious activity and generates alerts.

Objective

The goal of this lab is to simulate how an AI-powered campus safety system could work in a real-world setting.

This notebook will:

- create sample campus sensor data
- analyze activity based on time and location
- detect suspicious behavior
- trigger alerts
- generate an incident report
- visualize the results

```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Optional: make plots look a little cleaner
plt.rcParams["figure.figsize"] = (8, 5)
```

Step 1: Create Sample Sensor Data

This section creates simulated campus activity data.
The dataset includes:

- time of activity
- location
- whether movement was detected

This helps represent how a real campus safety system might receive input from sensors and cameras.

```
# Create simulated campus sensor data
data = {
    "time": ["10:00 AM", "2:00 PM", "2:00 AM", "11:30 PM", "3:15 AM", "1:45 PM"],
    "location": [
        "Main Entrance",
        "Library",
        "Restricted Lab",
        "Parking Lot",
        "Admin Office",
        "Student Center"
    ],
    "movement_detected": [1, 1, 1, 1, 1, 1]
}

# Convert dictionary to DataFrame
df = pd.DataFrame(data)

# Display dataset
df
```

	time	location	movement_detected
0	10:00 AM	Main Entrance	1
1	2:00 PM	Library	1
2	2:00 AM	Restricted Lab	1
3	11:30 PM	Parking Lot	1
4	3:15 AM	Admin Office	1
5	1:45 PM	Student Center	1

Next steps: Generate code with df New interactive sheet

Step 2: Apply AI Detection Logic

This section uses a simple rule-based model to classify activity as either:

- normal activity
- suspicious late-night activity
- suspicious activity in a restricted area

In this simulation, activity in restricted areas is always flagged as suspicious.
Activity that occurs during late-night hours, between 12:00 AM and 4:59 AM, is also treated as suspicious.

Even though this is a simple simulation, it helps demonstrate how an autonomous safety system can make decisions from incoming data.

This rule-based model simulates anomaly detection by identifying unusual activity based on time and location patterns.

```
# Function to detect suspicious activity
def detect_suspicious_activity(row):
    """
    Classifies activity based on time and location.

    Rules used in this simulation:
    - Activity between 12:00 AM and 4:59 AM is considered late-night suspicious activity
    - Any movement in a restricted area is considered suspicious
    - All other activity is marked as normal
    """

    # Rule 1: Restricted areas are always suspicious
    if "Restricted" in row["location"]:
        return "Suspicious Activity - Restricted Area"

    # Rule 2: Late-night activity based on AM times only
    time_str = row["time"].strip()
    if "AM" in time_str:
        hour = int(time_str.split(":")[0])
        if hour in [12, 1, 2, 3, 4]:
            return "Suspicious Activity - Late Night"

    return "Normal Activity"

# Apply detection function to each row
df["status"] = df.apply(detect_suspicious_activity, axis=1)

# Show updated dataset
df
```

	time	location	movement_detected	status
0	10:00 AM	Main Entrance	1	Normal Activity
1	2:00 PM	Library	1	Normal Activity
2	2:00 AM	Restricted Lab	1	Suspicious Activity - Restricted Area
3	11:30 PM	Parking Lot	1	Normal Activity
4	3:15 AM	Admin Office	1	Suspicious Activity - Late Night
5	1:45 PM	Student Center	1	Normal Activity

Next steps: Generate code with df New interactive sheet

Step 3: Trigger Alerts

This section filters the suspicious activities and displays alert messages.

In a real system, these alerts could be sent to:

- campus police
- a dashboard
- a mobile app
- an emergency notification system

```
# Filter only suspicious activity
alerts = df[df["status"].str.contains("Suspicious")]

# Display alerts
if not alerts.empty:
    print("ALERTS TRIGGERED\n")
    print(alerts[["time", "location", "status"]])
else:
    print("No suspicious activity detected.")
```

```
ALERTS TRIGGERED

      time        location                                status
2  2:00 AM  Restricted Lab  Suspicious Activity - Restricted Area
4  3:15 AM    Admin Office       Suspicious Activity - Late Night
```

Step 4: Generate an Incident Report

This section creates a short text-based incident report based on the suspicious activity detected.

This simulates how AI can help summarize events for faster review by officers or staff.

```
# Function to generate an AI-style incident report
def generate_report(alerts_df):
    """
    Creates a text summary of suspicious incidents.
    """

    if alerts_df.empty:
        return "No incidents detected. Campus activity appears normal."

    report = "AI Incident Report\n"
    report += "-" * 40 + "\n"

    for _, row in alerts_df.iterrows():
        report += f"Time: {row['time']}\n"
        report += f"Location: {row['location']}\n"
        report += f"Status: {row['status']}\n"
        report += "-" * 40 + "\n"

    return report

# Print report
print(generate_report(alerts))
```

```
AI Incident Report
----------------------------------------
Time: 2:00 AM
Location: Restricted Lab
Status: Suspicious Activity - Restricted Area
----------------------------------------
Time: 3:15 AM
Location: Admin Office
Status: Suspicious Activity - Late Night
----------------------------------------
```

Step 5: Summary Statistics

This section gives a quick summary of how many total activities were recorded and how many were flagged as suspicious.

This is useful for reviewing system performance at a glance.

```
# Count total activities and suspicious activities
total_activities = len(df)
total_alerts = len(alerts)
normal_activities = len(df[df["status"] == "Normal Activity"])

print(f"Total Activities Recorded: {total_activities}")
print(f"Normal Activities: {normal_activities}")
print(f"Suspicious Activities: {total_alerts}")
```

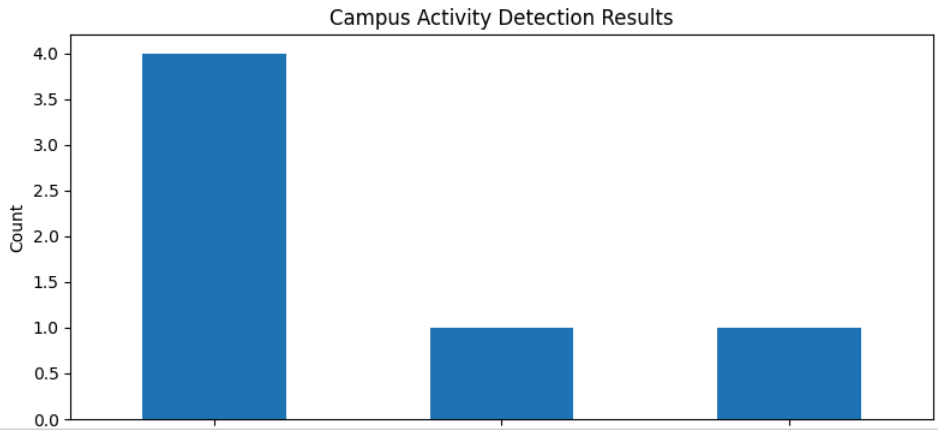
```
Total Activities Recorded: 6
Normal Activities: 4
Suspicious Activities: 2
```

Step 6: Visualize the Results

This bar chart compares the number of normal activities and suspicious activities detected by the system.

```
# Create a bar chart of activity classifications
status_counts = df["status"].value_counts()

ax = status_counts.plot(kind="bar")
plt.title("Campus Activity Detection Results")
plt.xlabel("Activity Status")
plt.ylabel("Count")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```



Step 7: Final Dataset Review

This section shows the completed dataset with all classifications included.

```
# Display final dataset
df
```

	time	location	movement_detected	status
0	10:00 AM	Main Entrance	1	Normal Activity
1	2:00 PM	Library	1	Normal Activity
2	2:00 AM	Restricted Lab	1	Suspicious Activity - Restricted Area
3	11:30 PM	Parking Lot	1	Normal Activity
4	3:15 AM	Admin Office	1	Suspicious Activity - Late Night
5	1:45 PM	Student Center	1	Normal Activity

Next steps: Generate code with df New interactive sheet

Conclusion

In this lab, I created a simulated AI-powered campus safety system using Python.

The system was able to:

- process sample campus activity data
- detect suspicious behavior based on time and location
- trigger alerts
- generate an incident report
- visualize the final results

This project helped me better understand how AI, sensors, and decision-making logic can work together in a real-world safety system. It also showed how autonomous systems can support faster responses and improve monitoring in campus environments.

This simulation demonstrates how a simple rule-based system can represent the foundation of more advanced AI models.

Bonus: Export Results

This section saves the final dataset as a CSV file.
In a real system, this could be used for recordkeeping, reporting, or later review.

```
# Save final dataset to CSV
df.to_csv("campus_safety_results.csv", index=False)

print("File saved as campus_safety_results.csv")
```

```
File saved as campus_safety_results.csv
```